

[Home](#) › [My Courses](#) › [Risk Management](#) › [M6: Network Learning for Risk Management](#) › Lesson Notes

Lesson Notes

MODULE 6 | LESSON 2

HOW TO LEARN THE STRUCTURE OF A NETWORK

Reading Time	20 minutes
Prior Knowledge	Network topology, Kruskal's algorithm, Minimum Spanning Tree
Keywords	Chow-Liu algorithm

In the last lesson, we learned more about how to quantify and describe the existing structure of a network. In this lesson, we find out some basic approaches to go about learning what the optimal structure of a network is.

1. Learning How to Learn

In the last lesson, we discussed the simple properties of a network: the types of graphs, the distributions of degrees, cycles, and types of components. In this lesson, we go deeper by investigating its very structure at a deeper level: Before we can count degrees or connections—or even examine the connectedness of components—we need to learn where the connections are, or should be, in our network: This is structure learning, and of course when we say structure in this context, we almost always mean the dependency structure across variables in our dataset. This includes independence as well as conditional dependence (and conditional independence). In this lesson, we will examine the techniques of structure learning by picking up where we left off with the pgmpy library that we discussed and inspected in Lesson 4 of Module 5.

The Chow-Liu algorithm should result in “essentially the best tree structure that can be found”—which suggests we should spend some time getting familiar enough with it to use it ourselves to construct our own powerful Bayesian networks to use as risk management tools. This is, after all, the endgame for the rest of this course.

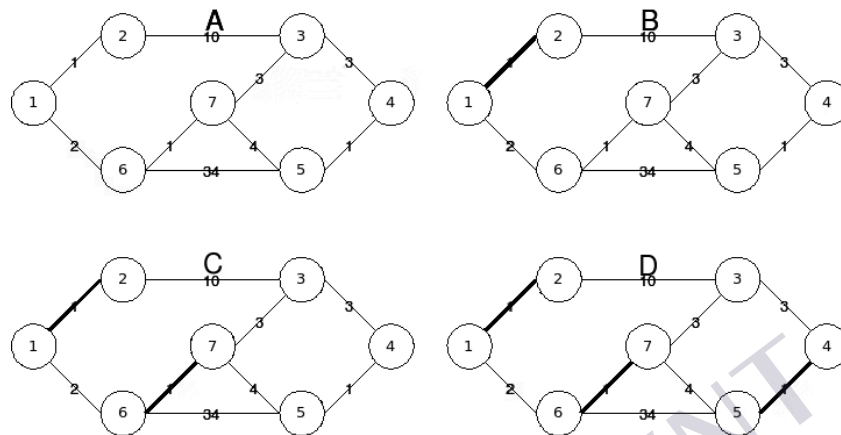
2. The Chow-Liu Algorithm: The Logical Background

To learn the structure of a directed acyclic graph (of which, a Bayesian network is an example), we need to “estimate statistically which node is connected to which node” (Koski 32). Based on what we already know about mutual information—that it quantifies the degree of dependence between two variables—it makes an obvious candidate for helping us identify the structure in our variables’ data. Since an undirected acyclic graph must have a unique path between any two nodes, there

exists a spanning tree of the graph (Koski 36), which you might already know from our previous discussion of spanning trees in Portfolio Management.

Figure 1 is presented here to refresh your memory about how spanning trees work: It shows the value associated with each possible arc in the graph, and the step-by-step result of working through the algorithm (iteratively choosing the smallest value as a new connection such that a cycle is not formed).

Figure 1: Minimum Spanning a Network



Source: Thedsadude. "File:Dsa minimum spanning tree kruskal 1.png." *Wikimedia*, https://commons.wikimedia.org/wiki/File:Dsa_minimum_spanning_tree_kruskal_1.png.

The spanning tree will help tell us which connections between variables stay and which are eliminated, which is very important. However, the other thing we know about mutual information is that it is symmetric; after all, it has mutual in its name. So mutual information alone will not provide the direction or directedness for us to derive our instance of a directed acyclic graph. To that end, we need to choose a node to serve as the root, which in fact is technically arbitrary for this purpose, as hard as that may be to believe. See figure 2 where both node 2 and node 3 serve as valid root nodes. See also figure 3: Now, since the choice is otherwise arbitrary, we should choose the one that is consistent with logic and our intuition. In this case, it makes sense that we align the direction of the arcs with the direction of time, such that the Tokyo Stock Exchange (TSE) opening precedes the TSE closing, the TSE opening also precedes the start of London trading (FTSE), and these precede the New York Stock Exchange (NYSE) opening, etc. By the way, when there is only one possible root, the tree is called proper.

Figure 2: Same Dependence Structures with Different Root Nodes

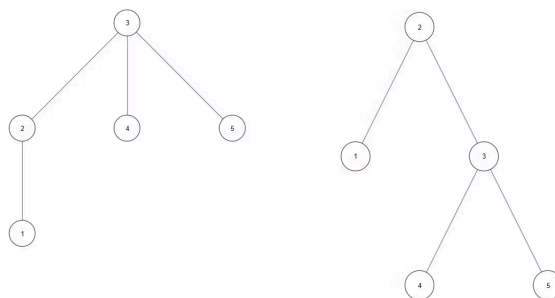
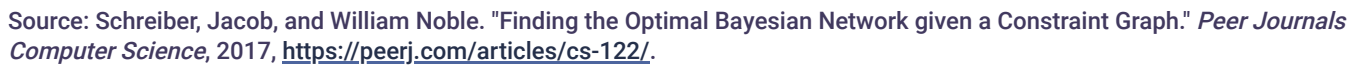


Figure 3: A Directed Acyclic Graph with an Obvious, though not Proper, Root



In the pgmpy library, and as we just saw, a Chow-Liu tree is formed in three steps:

- (pgmpy TreeSearch).

The function `estimate` sets about to do exactly this, where you can see the default assignments `"estimator_type="chow-liu"` and `"edge_weights_fn="mutual_info"`.

The mutual information calculation is handled by a call to the relevant sklearn library. Most likely pgmpy depends on the Kruskal algorithm or a very similar one, which you recall is a spanning tree algorithm, was used in Portfolio Management in its *Minimum* Spanning Tree form. The same idea lends itself to *Maximum* Spanning, where we want to connect the variables (represented by nodes) with the highest (not lowest) mutual information.

4. Fast vs. Correct

In one contest between the Chow-Liu algorithm and others, with respect to “Time to Learn” (especially for higher numbers of variables) as well as its performance, Schreiber comments that Chow-Liu performs “the worst” compared to the other techniques, but depending on the exact purposes to which you are putting your Bayesian network, this underperformance may be insignificant given the huge trade-off in computation time. One can easily imagine risk management scenarios where you need inference results that are just good enough, but they are needed quickly; just as one can imagine risk management scenarios where the inference needs to be as accurate as possible, and there is the luxury of time to work out the most correct results with the slowest techniques. Finally, one can easily imagine risk management data analysis needs on a spectrum between these two scenarios. What’s worse and more common, of course, is when the inference results need to be extremely accurate and immediately forthcoming.

5. Conclusion

In this lesson, we explored more of the functionality in the Sorobn library and discussed the importance of algorithms in learning complex network structures. In the next lesson, we read and discuss one of the great early algorithms for learning Bayesian network structure.

References

- Schreiber, Jacob, and William Noble. "Finding the Optimal Bayesian Network given a Constraint Graph." *Peer Journals Computer Science*, 2017, <https://peerj.com/articles/cs-122/>.
- Schreiber, Jacob. "Pomegranate." *GitHub*. <https://github.com/jmschrei/pomegranate>.
- Koski, Timo. "Lectures on Statistical Learning Theory for Chow-Liu Trees." The 32nd Finnish Summer School on Probability Theory, 2010, <https://people.kth.se/~tjtkoski/chowliulect.pdf>.
- Halford, Max. "Sorobn." *GitHub*. <https://github.com/MaxHalford/sorobn>.
- Pgmpy. "Source Code for pgmpy.estimators.TreeSearch." https://pgmpy.org/_modules/pgmpy/estimators/TreeSearch.html#TreeSearch

